

Genomic-RAG Pipeline for RARS1 Variant Analysis

The problem

Research on the RARS1 gene is scattered across hundreds of papers and abstracts on PubMed. If you wanted to know what a specific variant actually does, you had to read through them one by one. That's slow, and it's easy to miss something important.

There was no way to search this literature directly and get a straight, evidence-backed answer. I built one: a pipeline that turns the literature into a searchable vector database and answers natural-language questions using only what's actually in the source abstracts.

What I built

Data collection. Searching PubMed for just "RARS1" barely returned anything — 12 articles, not enough to work with. I built four query variations (RARS1, arginyl-tRNA synthetase, ArgRS, RARS, cytoplasmic), pulled 200 articles per query, then ran a custom filter on titles and abstracts to narrow it down to the 50 most relevant.

Chunking. Standard chunking tools split HGVS mutation names right down the middle — c.5A>G would come out as two meaningless fragments. I wrote a custom regex-based chunker that keeps these intact.

Vector database. ChromaDB, with all-MiniLM-L6-v2 for embeddings. It's fast, runs locally, and handles the semantic relationships in biological text well.

LLM. I started with OpenAI, but rate limits, API errors, and inconsistent JSON output made it unreliable for the structured variant/disease data I needed. I switched to Qwen2.5-3B running locally on Ollama. Zero API cost, and fully reproducible since nothing depends on an external service.

Guardrail against hallucination. If a question falls outside what the retrieved context actually supports, the system says "I do not know" instead of guessing. Every answer has to come with a PMID source.

What didn't work

- **Narrow search.** My first PubMed search using just "RARS1" returned only 12 articles — nowhere near enough. I had to broaden the query strategy significantly before the system had anything real to work with.
- **OpenAI as the backbone.** Rate limits and inconsistent structured output slowed the whole project down and made it hard to trust. Moving to a local model fixed both the cost and the stability problem.
- **Standard chunking.** LangChain's default chunking broke mutation names like c.2T>C (p.Met1Thr) in half, which meant the system couldn't recognize them as genetic data at all. I had to write my own chunking logic to keep that structure intact.

What it actually does

I tested it two ways.

Hallucination tests — asking questions it shouldn't be able to answer:

- "COVID-19 symptoms?" → "I do not know based on the retrieved literature." Pass.
- "Diabetes treatment?" → Same. Pass.
- "RARS2 variants?" → It correctly identified RARS2 but refused to connect it to RARS1. Pass.

Real query test — asking what it's actually for:

- Query: "List the genetic variants of RARS1 and their associated clinical phenotypes and diseases with PMIDs."
- Result: for the c.5A>G variant, it correctly linked phenotypes like hypomyelination and nystagmus to Pelizaeus-Merzbacher-like disease, with PMID:33515434 cited for the claim, in structured JSON.

It doesn't just generate plausible-sounding text. It stays inside the evidence and shows its source every time.

What I'd change next

1. **Hybrid search.** Right now it's pure vector search. For specific terms like mutation names, keyword search (BM25) would do better. Combining vector and keyword search would improve accuracy on exact matches.
2. **Reranking.** I currently take the top 50 chunks by vector similarity, but the most useful one for a given question isn't always the highest-scoring one. Adding a reranking layer (like Cohere Rerank) would improve what actually reaches the model.
3. **Automated evaluation.** My testing so far has been manual — trick questions and spot checks. I'd bring in a framework like RAGAS to track faithfulness and relevance numerically, so I can measure not just whether it works, but how consistent it is.

About Me

I'm a backend AI engineer. I build AI pipelines that turn messy, unstructured data into something reliable enough to use in production — whether that's genomic research or financial fraud detection. My focus is closing the gap between a model that works in a notebook and a system that works in the real world.

Contact

- GitHub: github.com/caglaeren
- LinkedIn: linkedin.com/in/cagla-eren